

UNIVERSIDAD NACIONAL EXPERIMENTAL DE TÁCHIRA
VICERRECTORADO ACADÉMICO
DECANATO DE DOCENCIA
DEPARTAMENTO DE INGENIERÍA INFORMÁTICA
REDES NEURONALES

Actividad 2 - INFERENCIA DIFUSA

Autor:

Juan Diego Paredes Gámez V-31.357.791

Sección: 01

San Cristóbal, enero de 2025

1. Introducción y Objetivos

La presente actividad tiene como objetivo implementar un sistema de inferencia difusa (Fuzzy Logic) para determinar la velocidad de un ventilador basándose en las condiciones operativas de un computador. Se busca complementar los conceptos teóricos estudiados en la materia de Redes Neuronales, aplicando lógica computacional para la toma de decisiones basada en reglas lingüísticas.

Para ello, se utiliza el lenguaje de programación Python y la librería `scikit-fuzzy`, definiendo variables lingüísticas, funciones de pertenencia trapezoidales y evaluando el comportamiento del sistema ante diferentes escenarios. Finalmente, se realiza una modificación a la base de reglas para analizar cómo afecta la lógica de los operadores (AND/OR) en el resultado final.

2. Definición del Sistema

Siguiendo los lineamientos de la evaluación, se definieron tres variables de entrada y una de salida con sus respectivos universos de discurso y funciones de membresía trapezoidales (TRAPMF):

Variables de Entrada:

1. **Temperatura (TEMPERATURATARMAD):** Rango [25, 70]. Se divide en 3 conjuntos difusos: T1, T2 y T3.
2. **Velocidad Procesador (VELOCIDAD PROC):** Rango [1.2, 3.2]. Se divide en 3 conjuntos: V1, V2 y V3.
3. **Actividad Disco Duro (ACTIVIDAD DISCDUR):** Rango [0, 100]. Se divide en 3 conjuntos: A1, A2 y A3.

Variable de Salida:

- **Ventilador (VENTILADOR):** Rango [0, 1]. Se divide en dos estados: OFF y ON.

```
# Temperatura: [25, 70]
temperatura = ctrl.Antecedent(np.arange(25, 71, 1), 'temperatura')
# Velocidad Proc: [1.2, 3.2]
velocidad = ctrl.Antecedent(np.arange(1.2, 3.3, 0.1), 'velocidad')
# Actividad Disco: [0, 100]
actividad = ctrl.Antecedent(np.arange(0, 101, 1), 'actividad')
# Salida Ventilador: [0, 1]
ventilador = ctrl.Consequent(np.arange(0, 1.1, 0.1), 'ventilador')
```

3. Base de Reglas Original

El sistema base se rige por las siguientes reglas lógicas proporcionadas en el planteamiento del problema:

1. **Regla 1:** IF (T1) AND (V1) AND (A1) THEN (OFF).

2. **Regla 2:** IF (T3) OR (V3) OR (A3) THEN (ON).

3. **Regla 3:** IF (T2) AND (V2) AND (A2) THEN (ON).

4. Resultados de la Simulación

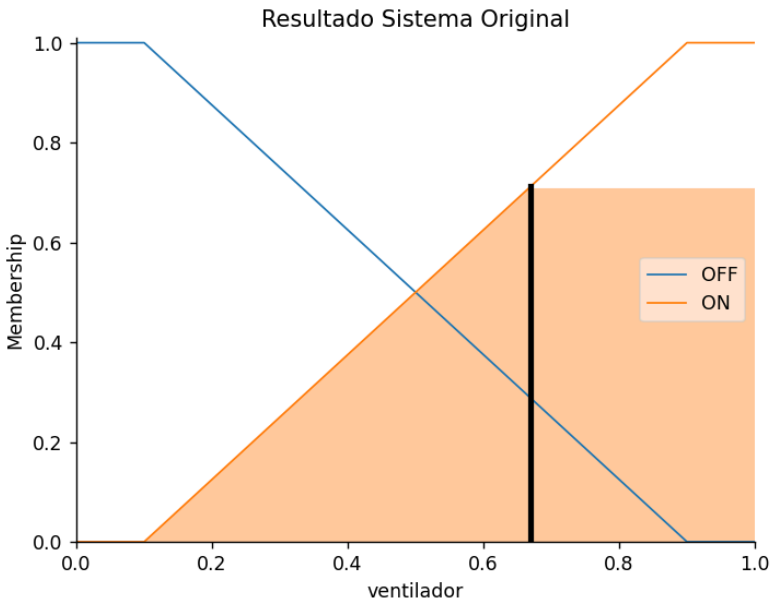
(Sistema Original)

Para validar el sistema, se ingresaron los siguientes valores de prueba intermedios:

- Temperatura: 55
- Velocidad: 2.5
- Actividad: 60

Estos valores activan parcialmente los conjuntos intermedios (T2, V2, A2). Al computar el sistema original (que usa operador AND en la Regla 3), se obtuvo el siguiente resultado:

Salida Ventilador: `0.6700`



El gráfico muestra cómo el método del Centroide calcula el punto de equilibrio del área activada. Debido al uso del operador AND (intersección) en las reglas activas, el "corte" de la función de membresía es más bajo, limitando la fuerza de la salida.

5. Modificación y Análisis Comparativo

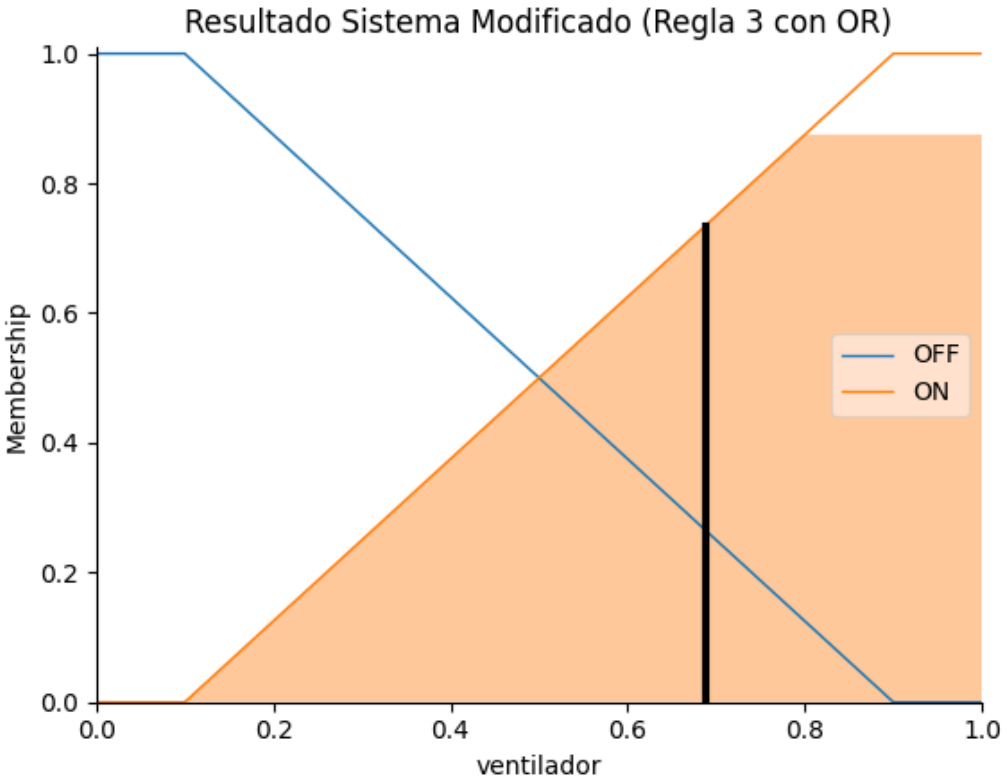
Como parte de la actividad, se solicitó modificar al menos una regla y repetir los pasos anteriores.

Modificación realizada: Se alteró la **Regla 3**. Originalmente utilizaba el operador **AND** (intersección/mínimo), y se cambió por el operador **OR** (unión/máximo).

- *Original:* IF T2 **AND** V2 **AND** A2 THEN ON.
- *Modificada:* IF T2 **OR** V2 **OR** A2 THEN ON.

Al ejecutar el sistema con las mismas entradas (Temp=55, Vel=2.5, Act=60), se obtuvo un nuevo resultado:

Salida Ventilador (Modificado): `0.6879`



Análisis de la Variación

Se observa un incremento en la salida del ventilador (de 0.6700 a 0.6879). Esto es consistente con la teoría de la lógica difusa:

1. **Operador AND:** Restringe la activación al valor de verdad **mínimo** de las premisas. Es más "estricto", por lo que el área resultante bajo la curva del conjunto 'ON' fue menor (más baja).

2. **Operador OR:** Permite la activación basada en el valor de verdad **máximo**. Es más "permisivo". Esto provocó que el conjunto 'ON' se activara con mayor intensidad (mayor altura en la gráfica), desplazando el centroide hacia la derecha y resultando en una salida numérica mayor.

```
--- INICIANDO SISTEMA DE INFERENCIA DIFUSA ---

--- EJECUTANDO SISTEMA ORIGINAL ---
Entradas: Temp=55, Vel=2.5, Act=60
Salida Ventilador (Original): 0.6700

--- EJECUTANDO SISTEMA MODIFICADO ---
Modificación: Se cambia la Regla 3 para que sea más sensible (usando OR en vez de AND).
Salida Ventilador (Modificado): 0.6879

--- FIN DEL PROCESO ---
```

6. Conclusión

A través de esta práctica se logró implementar exitosamente un sistema de inferencia difusa utilizando funciones de pertenencia trapezoidales. Se comprobó que el diseño de las reglas y la elección de los operadores lógicos (AND vs OR) tienen un impacto directo en la agresividad o suavidad de la respuesta del sistema de control. La herramienta computacional permitió visualizar claramente el proceso de defuzzificación por centroide, validando la lógica planteada en los objetivos de la evaluación.

Anexos

Para la correcta ejecución del algoritmo de inferencia difusa, se utilizó el lenguaje de programación **Python** (versión 3.x). Es necesario instalar las siguientes librerías externas mediante el gestor de paquetes `pip`.

Comando de instalación:

```
pip install numpy scikit-fuzzy matplotlib scipy networkx
```

Descripción de las librerías utilizadas:

- **NumPy:** Para el manejo de arreglos numéricos y cálculos matemáticos de alto rendimiento.

- **Scikit-Fuzzy:** Librería especializada que contiene los algoritmos de lógica difusa (fuzzificación, inferencia y desfuzzificación).

- **Matplotlib:** Utilizada para la generación de gráficas visuales de las funciones de membresía y resultados.

- **SciPy y NetworkX:** Dependencias necesarias para el funcionamiento interno de los algoritmos de control y visualización.

```
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl
import matplotlib.pyplot as plt

print("--- INICIANDO SISTEMA DE INFERENCIA DIFUSA ---")


#1 DEFINICIÓN DE VARIABLES (COMÚN)

# Temperatura: [25, 70]
temperatura = ctrl.Antecedent(np.arange(25, 71, 1), 'temperatura')
# Velocidad Proc: [1.2, 3.2]
velocidad = ctrl.Antecedent(np.arange(1.2, 3.3, 0.1), 'velocidad')
```

```
# Actividad Disco: [0, 100]
actividad = ctrl.Antecedent(np.arange(0, 101, 1), 'actividad')
# Salida Ventilador: [0, 1]
ventilador = ctrl.Consequent(np.arange(0, 1.1, 0.1), 'ventilador')


#2 FUNCIONES DE MEMBRESÍA (TRAPEZOIDALES)


# --- Temperatura ---
temperatura['T1'] = fuzz.trapmf(temperatura.universe, [4.75, 22.75,
27.25, 45.25])
temperatura['T2'] = fuzz.trapmf(temperatura.universe, [27.25,
45.25, 49.75, 67.75])
temperatura['T3'] = fuzz.trapmf(temperatura.universe, [49.75,
67.75, 72.25, 90.25])

# --- Velocidad ---
velocidad['V1'] = fuzz.trapmf(velocidad.universe, [0.3, 1.1, 1.3,
2.1])
velocidad['V2'] = fuzz.trapmf(velocidad.universe, [1.3, 2.1, 2.3,
3.1])
velocidad['V3'] = fuzz.trapmf(velocidad.universe, [2.3, 3.1, 3.3,
4.1])

# --- Actividad ---
actividad['A1'] = fuzz.trapmf(actividad.universe, [-45, -5, 5, 45])
actividad['A2'] = fuzz.trapmf(actividad.universe, [5, 45, 55, 95])
actividad['A3'] = fuzz.trapmf(actividad.universe, [55, 95, 105,
145])

# --- Ventilador (Salida) ---
ventilador['OFF'] = fuzz.trapmf(ventilador.universe, [-0.9, -0.1,
0.1, 0.9])
ventilador['ON'] = fuzz.trapmf(ventilador.universe, [0.1, 0.9, 1.1,
1.9])


#3 SISTEMA ORIGINAL

print("\n--- EJECUTANDO SISTEMA ORIGINAL ---")
```

```
# Regla 1: 1 1 1, 1 (AND) -> Si T1 y V1 y A1 entonces OFF
rule1 = ctrl.Rule(temperatura['T1'] & velocidad['V1'] &
actividad['A1'], ventilador['OFF'])

# Regla 2: 3 3 3, 2 (OR, según nota :2) -> Si T3 o V3 o A3 entonces
ON
rule2 = ctrl.Rule(temperatura['T3'] | velocidad['V3'] |
actividad['A3'], ventilador['ON'])

# Regla 3: 2 2 2, 2 (AND, según nota :1) -> Si T2 y V2 y A2
entonces ON
rule3 = ctrl.Rule(temperatura['T2'] & velocidad['V2'] &
actividad['A2'], ventilador['ON'])

# Crear Sistema de Control Original
sistema_orig = ctrl.ControlSystem([rule1, rule2, rule3])
simulacion_orig = ctrl.ControlSystemSimulation(sistema_orig)

# Entradas de prueba (Valores arbitrarios para poner a prueba el
sistema)

input_temp = 55
input_vel = 2.5
input_act = 60

simulacion_orig.input['temperatura'] = input_temp
simulacion_orig.input['velocidad'] = input_vel
simulacion_orig.input['actividad'] = input_act

try:
    simulacion_orig.compute()
    resultado_orig = simulacion_orig.output['ventilador']
    print(f"Entradas: Temp={input_temp}, Vel={input_vel},
Act={input_act}")
    print(f"Salida Ventilador (Original): {resultado_orig:.4f}")

    # Visualizar
    ventilador.view(sim=simulacion_orig)
    plt.title("Resultado Sistema Original")
    plt.show()
except Exception as e:
    print(f"Error en sistema original: {e}")
```

```python
#4 SISTEMA MODIFICADO

print("\n--- EJECUTANDO SISTEMA MODIFICADO ---")
print("Modificación: Se cambia la Regla 3 para que sea más sensible
(usando OR en vez de AND).")


# Nueva Regla 3: Si T2 O V2 O A2 entonces ON (Antes era Y)
rule3_mod = ctrl.Rule(temperatura['T2'] | velocidad['V2'] |
actividad['A2'], ventilador['ON'])

# IMPORTANTE: Creamos un NUEVO sistema con la regla modificada
sistema_mod = ctrl.ControlSystem([rule1, rule2, rule3_mod])
simulacion_mod = ctrl.ControlSystemSimulation(sistema_mod)

# Usamos las MISMAS entradas para poder comparar justamente
simulacion_mod.input['temperatura'] = input_temp
simulacion_mod.input['velocidad'] = input_vel
simulacion_mod.input['actividad'] = input_act

try:
    simulacion_mod.compute()
    resultado_mod = simulacion_mod.output['ventilador']
    print(f"Salida Ventilador (Modificado): {resultado_mod:.4f}")

    # Visualizar comparación
    ventilador.view(sim=simulacion_mod)
    plt.title("Resultado Sistema Modificado (Regla 3 con OR)")
    plt.show()
except Exception as e:
    print(f"Error en sistema modificado: {e}")

print("\n--- FIN DEL PROCESO ---")
```